MEMBER 1 · <NAME-1> · <ROLL-1>

23I501 – DESIGN AND ANALYSIS OF ALGORITHMS · BATCH 14

Valid ECU Feature Configurations
Automotive software · Backtracking & pairwise testing

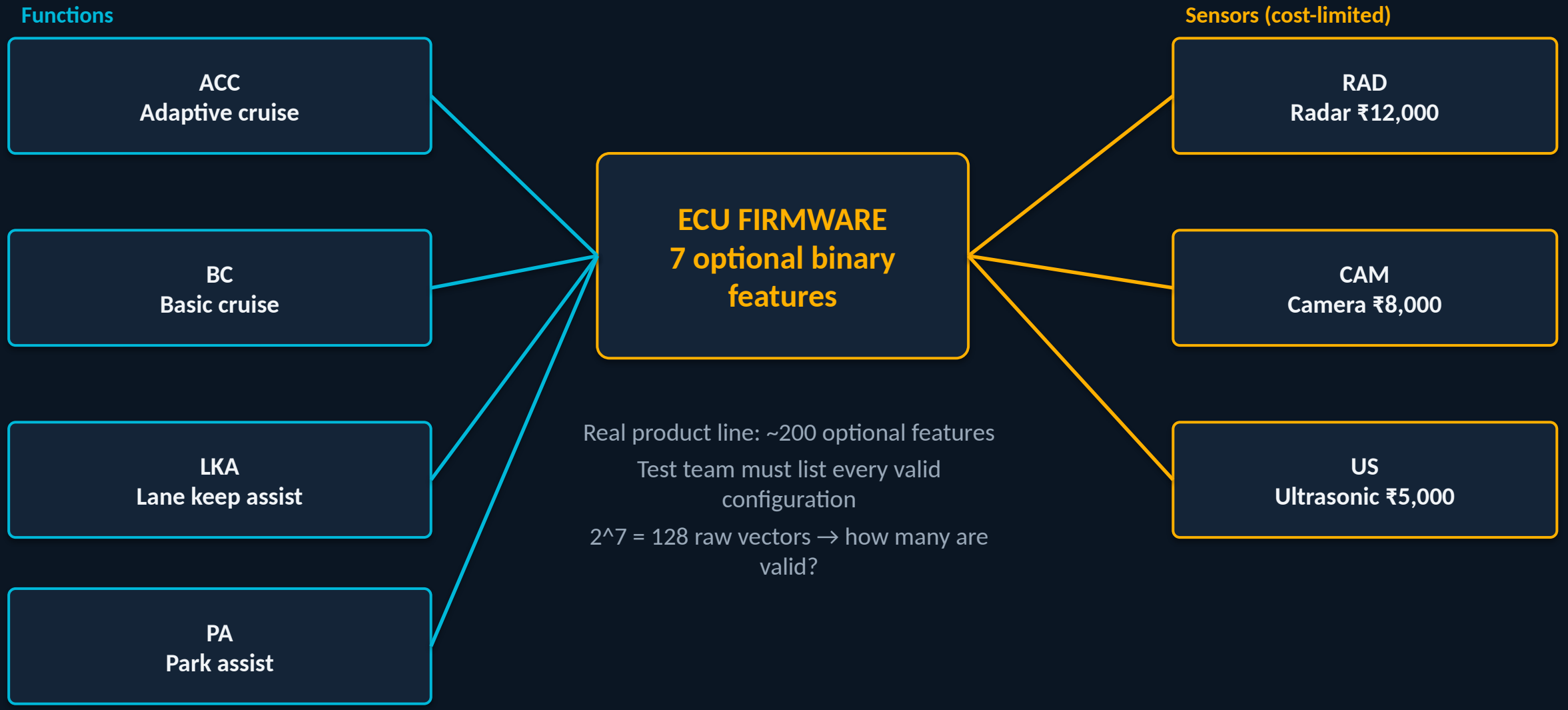
PSG College of Technology · Department of Information Technology

Member	Name	Roll No.	Part
Member 1	<NAME-1>	<ROLL-1>	Introduction & ECU problem
Member 2	<NAME-2>	<ROLL-2>	CSP modelling
Member 3	<NAME-3>	<ROLL-3>	Backtracking & rejected techniques
Member 4	<NAME-4>	<ROLL-4>	Configurations, trace, ordering, data structures
Member 5	<NAME-5>	<ROLL-5>	Complexity, benchmark, worst case, pairwise

The ECU variant problem

MEMBER 1 · <NAME-1> · <ROLL-1>

One firmware image, many vehicle variants - every valid variant must be tested



Real product line: ~200 optional features

Test team must list every valid configuration

2^7 = 128 raw vectors → how many are valid?

Problem statement and rules

MEMBER 1 · <NAME-1> · <ROLL-1>

Rule	Meaning
ACC requires RAD	adaptive cruise needs radar
LKA requires CAM	lane keep needs camera
PA requires US	park assist needs ultrasonic
ACC excludes BC	never both cruise types
ACC or BC present	at least one cruise function
Sensor cost ≤ ₹20,000	RAD 12k + CAM 8k + US 5k budget

Formal task
Input: 7 binary features + constraints
Output: ALL x ∈ {0,1}^7 satisfying them
Objective: none - completeness
Class: ENUMERATION (all-solutions CSP)

Why enumerate, not optimise? Each valid variant needs testing, so none may be dropped.

Why not just brute-force? 2^7 is tiny, but 200 features → 2^200 ≈ 1.6×10^60: we need pruning and, finally, pairwise sampling.

Answer (verified by two programs): 19 valid configurations.

CSP model: variables and domains

MEMBER 2 · <NAME-2> · <ROLL-2>

CSP = (X, D, C)

Variable	Meaning	Domain	Cost if 1
ACC	Adaptive Cruise Control	{0,1}	-
BC	Basic Cruise	{0,1}	-
LKA	Lane Keep Assist	{0,1}	-
PA	Park Assist	{0,1}	-
RAD	Radar	{0,1}	₹12,000
CAM	Camera	{0,1}	₹8,000
US	Ultrasonic sensor	{0,1}	₹5,000

Search space
|D|^n = 2^7 = 128
complete vectors

Valid: 19
(14.8 %)

Unary constraints: only the domains {0,1}

Feature variables: ACC, BC, LKA, PA · Sensor variables: RAD, CAM, US. A configuration = one value per variable.

Constraints and their classification

MEMBER 2 · <NAME-2> · <ROLL-2>

#	Constraint	Formal	Class
C0	domains	$x \in \{0,1\}$	UNARY
C1	ACC requires RAD	$ACC \Rightarrow RAD$	BINARY
C2	LKA requires CAM	$LKA \Rightarrow CAM$	BINARY
C3	PA requires US	$PA \Rightarrow US$	BINARY
C4	ACC excludes BC	$\neg(ACC \wedge BC)$	BINARY
C5	ACC or BC present	$ACC \vee BC$	BINARY (at-least-1 over a group)
C6	sensor cost ≤ ₹20,000	$12000 \cdot RAD + 8000 \cdot CAM + 5000 \cdot US \leq 20000$	GLOBAL (3 vars, linear)

C6 is monotone: costs ≥ 0, so a partial sum above ₹20,000 can never recover → safe to prune early.

C6 forbids exactly $RAD \wedge CAM \wedge US$ (₹25,000). RAD+CAM = ₹20,000 is allowed.

Constraint graph

MEMBER 2 · <NAME-2> · <ROLL-2>

Edges = binary constraints; shaded group = global cost constraint



Green: ⇒ (dependency) Red: ACC–BC excludes + at-least-one (exactly one cruise function)

Backtracking with incremental pruning

MEMBER 3 · <NAME-3> · <ROLL-3>

```
Search(d, cost):
  if d = n: output A; return
  v ← order[d]
  for val in {0,1}:
    A[v] ← val;  c' ← cost + costOf(v)·val
    if c' ≤ budget and no constraint closing
       at depth d is violated:
         Search(d+1, c')          // else PRUNE
  unset A[v]                       // backtrack
```

Why it works (prunable state space)

Constraint checked the moment its last variable is assigned.

Lemma: a violated constraint over assigned variables stays violated in every extension; partial cost only grows.

⇒ whole subtree (2^(n-d-1) leaves) is skipped without losing a solution.

Both variable orders implemented: sensors-first, features-first.

Real counts (ECU): features-first 66 calls / 94 assignments examined; sensors-first 67 calls / 96.

Full tree would be 255 nodes, 254 assignments, 128 leaves.

Why the other four techniques fail

MEMBER 3 • <NAME-3> • <ROLL-3>

Technique	Concrete reason (evidence)
Divide & Conquer	Halves share constraints (ACC–RAD, LKA–CAM, PA–US, cost). Split {ACC,BC,LKA,PA}|{RAD,CAM,US}: 8 × 7 = 56 merge pairs to find 19 valid; T(n)=2T(n/2)+Θ(2^n)=Θ(2^n).
Greedy	Irrevocable choice yields ONE configuration; answer has 19 (14 with BC, 5 with ACC). Choosing ACC first can never reach the BC ones. No objective ⇒ no exchange argument.
Dynamic Programming	Can only COUNT (19): state = (index, cost, pending obligations) and pending dependencies form a subset → up to 2^k states. Listing needs re-walking the table = backtracking.
Branch & Bound	No objective to bound; every valid leaf is required. Min-cost solution is {BC} (₹0); bounding by it would prune the other 18 solutions. Only feasibility bounds are valid = backtracking's prune.

Exhaustive 2^n generate-and-test is the rejected implementation compared experimentally (same 19 solutions).

Correctness: soundness, completeness, uniqueness

MEMBER 3 · <NAME-3> · <ROLL-3>

Soundness

Leaf reached ⇒ every constraint was checked with full scope assigned, and final running cost = total cost ≤ ₹20,000. So output ⊆ valid.

Completeness

Invariant: every prefix of a valid configuration passes the checks (induction on depth). A check fails only if a constraint over assigned variables is violated or cost > budget; by the lemma that makes the configuration invalid. So valid ⊆ output.

Uniqueness

Two leaves differ at a first depth where one took 0, the other 1, in the same call. Each configuration ⇔ exactly one root-to-leaf path ⇒ no duplicates.

Empirical check: exhaustive search returns exactly the same 19 configurations; benchmark asserts equality for all sizes n = 7…21.

All 19 valid configurations

MEMBER 4 · <NAME-4> · <ROLL-4>

Columns: ACC BC LKA PA RAD CAM US (1 = enabled) · verified by backtracking (both orders) and exhaustive search

#	Config	#	Config
1	0100000 {BC}	11	0110010 {BC,LKA,CAM}
2	0100001 {BC,US}	12	0110011 {BC,LKA,CAM,US}
3	0100010 {BC,CAM}	13	0110110 {BC,LKA,RAD,CAM}
4	0100011 {BC,CAM,US}	14	0111011 {BC,LKA,PA,CAM,US}
5	0100100 {BC,RAD}	15	1000100 {ACC,RAD}
6	0100101 {BC,RAD,US}	16	1000101 {ACC,RAD,US}
7	0100110 {BC,RAD,CAM}	17	1000110 {ACC,RAD,CAM}
8	0101001 {BC,PA,US}	18	1001101 {ACC,PA,RAD,US}
9	0101011 {BC,PA,CAM,US}	19	1010110 {ACC,LKA,RAD,CAM}
10	0101101 {BC,PA,RAD,US}		

Examined vs 2^7

Features-first:
66 calls · 94 assignments
29 pruned

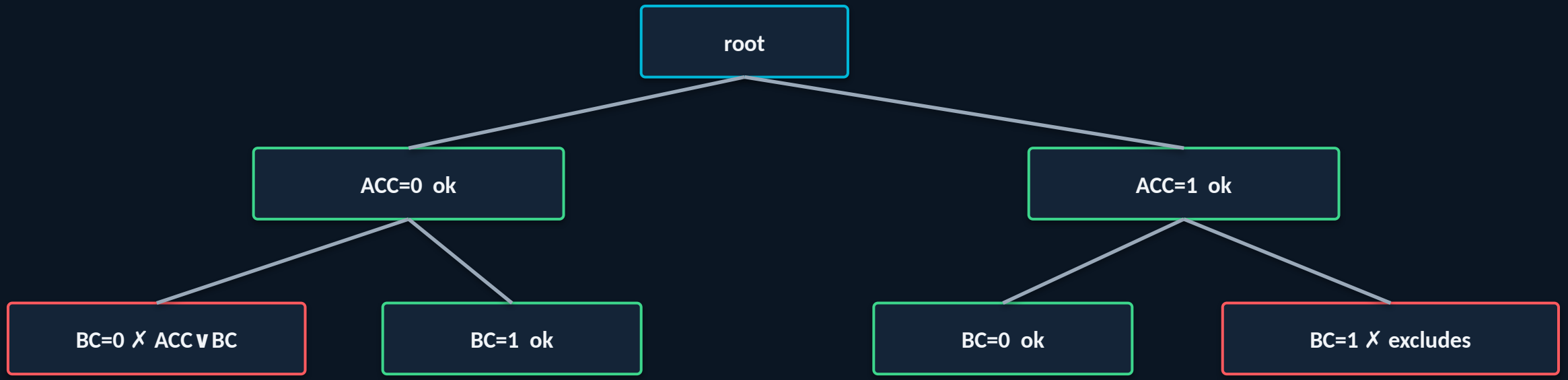
Sensors-first:
67 calls · 96 assignments
30 pruned

Full tree: 254 assignments
128 leaves

Execution trace (features-first, top levels)

MEMBER 4 · <NAME-4> · <ROLL-4>

From the actual trace: python main_solution.py --trace (148 lines)



… LKA, PA, RAD, CAM, US levels continue; sample prunes deeper in the same trace:

Partial config	Verdict	Reason
ACC=1,BC=0,LKA=0,PA=0,RAD=0	PRUNE (d5)	ACC ⇒ RAD violated
BC=1,…,RAD=1,CAM=1,US=1	PRUNE (d7)	cost 25,000 > 20,000
BC=1,PA=1,…,US=0	PRUNE (d7)	PA ⇒ US violated
BC=1,LKA=1,…,CAM=0	PRUNE (d6)	LKA ⇒ CAM violated

Variable ordering and constraint data structures

MEMBER 4 · <NAME-4> · <ROLL-4>

Statistic	Sensors-first	Features-first
Recursive calls	67	66
Assignments examined	96	94
Pruned (cost / logic)	30 (1 / 29)	29 (8 / 21)
Prunes by depth	d3:1 d4:4 d5:10 d6:6 d7:9	d2:2 d5:4 d6:6 d7:17
Mean prune depth	5.60 (earlier)	6.17

Features-first: ACC/BC rule closes at depth 2, cutting two 2^5 subtrees → least total work.

Sensors-first: budget and requires close at depth 3-5 → prunes earlier on average.

Good order = close constraints as early as possible.

Structure	Check cost	Space	Notes
Bit vector (chosen)	O(1)/constraint, O(c) full	O(n/w + c)	handles budget; O(1) undo
Implication graph	O(n+E) full; propagate O(edges)	O(n+E)	no global cost
BDD	O(n) per check	O(|BDD|): measured 23 / 24 nodes	order-sensitive, exp. worst

Complexity and worst case

MEMBER 5 · <NAME-5> · <ROLL-5>

Case	Backtracking time	Evidence
Best	O(1) (first var contradicts); O(n·|S|) with no dead ends	output-sensitive
Practical	O(N·c_max + n·|S|), N = nodes visited	ECU: N = 66 vs 255
Worst	Θ(2^n)	unconstrained / late constraint
Space	O(n) stack	O(n|S|) if stored
Exhaustive	Θ(2^n · c)	128 checks at n = 7

n	instance	valid	calls	pruned
10	unconstrained	1,024	2,047	0
10	x0 ⇒ x9 (late)	768	1,791	256
15	unconstrained	32,768	65,535	0
15	x0 ⇒ x14 (late)	24,576	57,343	8,192

Constructed worst case

Constraints only between the FIRST and LAST variable of the order are detected at depth n: nothing is pruned earlier.

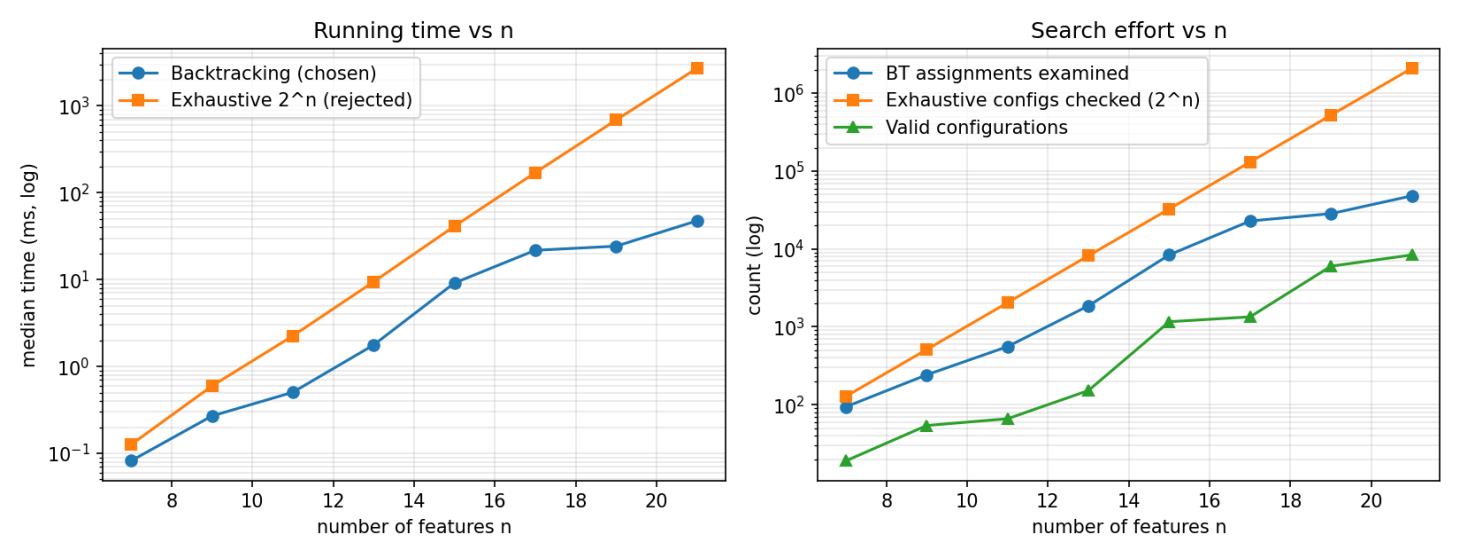
Measured at n = 15:
65,534 assignments of 65,534 possible (unpruned tree)

Θ(2^n) = size of the output itself

Benchmark: backtracking vs exhaustive

MEMBER 5 · <NAME-5> · <ROLL-5>

Median of 5 runs, CPython; n = 7 real ECU, n = 9…21 seeded synthetic instances



n	BT ms	Exh. ms	speed-up
7	0.084	0.124	1.5x
9	0.303	0.611	2.0x
11	0.955	2.458	2.6x
13	1.72	9.397	5.5x
15	8.958	41.816	4.7x
17	21.535	174.805	8.1x
19	30.896	711.291	23.0x
21	54.742	2965.309	54.2x

Exhaustive time ×≈4 per +2 features (2^n): 42 ms at n = 15 → 3.0 s at n = 21.

Backtracking time ∝ assignments examined (≈1 µs each), far below 2^n: 48,206 vs 2,097,152 at n = 21.

Limits: sub-ms noise at n = 7-9; exhaustive at n = 30 (>10^9 checks) not run; synthetic instances.

200 features → pairwise testing

MEMBER 5 · <NAME-5> · <ROLL-5>

Illustrative SYNTHETIC features F001…F200 (not the 7 real ECU features)

1 Feasible pairs

propagate 2 literals in implication graph; conflict-free ⇒ feasible

2 Greedy test

seed uncovered pair, extend each feature to the value covering most uncovered pairs

3 Verify

all tests valid; all feasible pairs covered

Model	Pairs (total)	Feasible	Tests	Covered
Real ECU (7)	84	78	6 (vs 19 valid)	78/78
Synthetic (200)	79,600	77,964	58	77,964/77,964

Cost: feasibility $O(n^2(n+E))$; greedy $O(T \cdot n \cdot (n+E) + T \cdot n^2)$.

58 tests is a greedy result, not proven optimal (unconstrained bound: 11).

Takeaways
Backtracking: exact, 19 configs
Exhaustive: same answer, 2^n
Pairwise: scales to 200
Ref: Batory SPLC 2005; NIST ACTS